МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт № 8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № 3

по курсу «Операционные системы и архитектура компьютера»

Управление потоками и синхронизация
Вариант 9. Определитель матрицы

Выполнил: А. А. Копылов
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 22.10.2026

Москва, 2026

1. Задание и алгоритм

Цель работы - освоить создание потоков ОС и синхронизацию доступа к общим данным. Требуется вычислить определитель матрицы на языке C, задавать максимальное число одновременно существующих потоков ключом запуска, показать потоки средствами ОС и исследовать ускорение и эффективность.

Использовано рекурсивное разложение Лапласа по первой строке. Полное раскрытие даёт ту же знакопеременную сумму произведений по перестановкам, что и определение определителя. Метод Гаусса в программе на C не используется.

```
D(A) = sum((-1)^j * A[0,j] * D(M[0,j]))
D([a]) = a
```

Вычеркивание первой строки и выбранного столбца реализовано функцией make_minor. Матрица хранится в одномерном массиве: элемент строки row и столбца col имеет индекс row*n + col. Размер входа: от 1 до 14. Элементы и результат имеют тип long double; для вещественных данных возможны ошибки округления. Верхняя граница размера ограничивает вход, но не гарантирует быстрое вычисление: сложность факториальная.

Верхние d уровней раскрываются заранее в prepare_tasks. Каждая задача содержит собственную оставшуюся матрицу и накопленный множитель со знаком. По умолчанию d=2. Число задач равно n!/(n-k)!, где k=min(d,n-1). Например, для n=5 получается 20 задач. При d=0 остаётся одна задача.

2. Организация потоков

Пул содержит min(лимит, число задач) потоков с учётом основного. Создаются только p-1 дополнительных потоков через pthread_create. Основной вызывает worker как обычную функцию, затем ожидает остальных через pthread_join. Поэтому -t 4 означает максимум четыре потока всего.

Задачи находятся в готовом массиве. Общий индекс next защищён одним pthread_mutex_t. В критической секции поток проверяет наличие задачи и увеличивает индекс. Рекурсивное вычисление выполняется после освобождения мьютекса. Каждый Worker хранит отдельную сумму result. Основной складывает суммы после завершения всех работников. Матрицы задач после запуска не изменяются, временные миноры находятся в стеке выполняющего потока.

```
lock(mutex)
index = next++  // only if a task is available
unlock(mutex)
local_sum += tasks[index].factor * determinant(minor)
```

3. Сборка и проверка

Основная проверка выполнена в Linux-контейнере: Linux 7.0.14-linuxkit, aarch64, GCC 13.3.0, glibc 2.39, восемь доступных логических CPU. Квота cpu.max: max 100000, то есть дополнительная квота cgroup не задана. Сборка CMake Release. Также выполнена сборка AppleClang 21 на macOS.

```
cmake -S src -B /tmp/lab3-build -DCMAKE_BUILD_TYPE=Release
cmake --build /tmp/lab3-build
ctest --test-dir /tmp/lab3-build --output-on-failure
/tmp/lab3-build/determinant -t 4 examples/matrix3.txt
```

Вход: размер n, затем n*n чисел по строкам. Файл необязателен: без него программа читает stdin. Пример matrix3.txt содержит матрицу [[6,1,1],[4,-2,5],[2,8,7]], ожидаемый определитель -306. Ключи: -t - лимит потоков; -d - глубина подготовки от 0 до 3; -w - пауза от 0 до 60 секунд для показа потоков. При замерах -w не используется.

Проверка	Результат
Точное сравнение	352 запуска: 22 матрицы, глубины 0..3, лимиты 1,2,4,8. Независимый эталон - сумма по перестановкам с рациональной арифметикой. Пройдены на macOS и Linux.
Граничные случаи	Матрицы 1x1, нулевая и единичная; совпадающие строки; нулевой ведущий элемент; дробные значения; случайные матрицы до 7x7.
Ошибки ввода	19 случаев: некорректные ключи, неполная матрица, лишние элементы, NaN/Inf, неверный размер, отсутствующий файл. Ошибки отклонены.
Память	ASan+UBSan: полный точный набор на macOS; в Linux базовый набор с detect_leaks=1. Сообщений об ошибках нет.
Потоки средствами ОС	Для -t 1,2,4,8 через ps и /proc зафиксированы соответственно 1,2,4,8 потоков.

В базовом наборе CTest - 16 сочетаний глубины и числа потоков, случай 1x1 с лимитом 256 и пять ошибочных входов. При наличии Python3 CMake дополнительно включает полный точный набор. В исходном Linux-образе Python отсутствовал: полный набор запускался отдельно в уже имевшемся Python-контейнере на том же Linux-бинарнике.

Ограничение проверки: ASan и UBSan не являются детекторами гонок данных. Отсутствие гонок обосновывается защитой next, неизменностью задач и раздельными результатами работников, читаемыми после join.

4. Исследование ускорения и эффективности

Использованы фиксированные матрицы размеров 5, 8, 10 и 11. Генератор Python Random(9000+n); элементы выбираются из {-3,-2,-1,1,2,3}. Входы сохранены в examples/benchmark*.txt. Каждый результат дополнительно проверен точным рациональным исключением в отдельном Python-драйвере. Это проверка результата, алгоритм программы на C остаётся рекурсивным.

Для каждого размера выполнен один прогрев и пять измерений на 1,2,4,8 потоках. В таблице - медианы. Порядок конфигураций в каждом повторе детерминированно перемешан. Глубина подготовки всегда 2. Замеры выполнены без strace и без паузы -w. CLOCK_MONOTONIC охватывает подготовку задач, выделение памяти пула, создание потоков, вычисления, join и итоговое суммирование; чтение входа, запуск процесса, печать и освобождение памяти исключены.

Ускорение S=T1/Tp. Эффективность E=S/p. Значение p учитывает основной поток. CSV с каждым измерением, минимумами и максимумами сохранены рядом с отчётом.

n	Потоки	Медиана, с	S	E, %
5	1	0.000030333	1.000	100.0
5	2	0.000127291	0.238	11.9
5	4	0.000193541	0.157	3.9
5	8	0.000306583	0.099	1.2
8	1	0.002071167	1.000	100.0
8	2	0.001198458	1.728	86.4
8	4	0.000780000	2.655	66.4
8	8	0.000710125	2.917	36.5
10	1	0.173937750	1.000	100.0
10	2	0.089658500	1.940	97.0
10	4	0.046059375	3.776	94.4
10	8	0.029569917	5.882	73.5
11	1	1.924820125	1.000	100.0
11	2	0.981516084	1.961	98.1
11	4	0.503170750	3.825	95.6
11	8	0.320440833	6.007	75.1

Короткие запуски 5x5 измеряются десятками микросекунд и чувствительны к шуму планировщика. Для них значения ускорения не следует интерпретировать как универсальные характеристики системы.

5. Объяснение результатов

На матрице 5x5 создание потоков и синхронизация занимают больше времени, чем экономится на вычислениях. При увеличении размера факториальный объём рекурсии быстро растёт, а число созданий потоков остаётся постоянным. Поэтому на 11x11 ускорение на восьми потоках составило около 6,01, эффективность - 75,1%.

Ускорение ограничивают последовательная подготовка задач, синхронизация выдачи, различное время завершения работников и особенности планирования CPU. При d=2 число задач равно n(n-1); при слишком большом лимите полезный параллелизм ограничен этим числом. Фактическое число потоков дополнительно ограничивается числом задач. Результаты относятся к указанной среде и не гарантируют такой же эффект на другом компьютере.

6. Демонстрация через strace и ps

```
sh src/tools/capture-linux.sh /tmp/lab3-build/determinant \
    /tmp/lab3-evidence
cat /tmp/lab3-evidence/ps-t4.txt
grep -E "clone|futex|exit" /tmp/lab3-evidence/strace-t4.txt
```

Скрипт запускает программу под strace -f, добавляя двухсекундную паузу перед работой. Во время паузы снимаются ps -L и /proc/PID/status. Эта пауза нужна только для наблюдения; полученное время не используется в исследовании ускорения.

```
PID     TID NLWP STAT COMMAND
    866     866    4 Sl   determinant
    866     870    4 Sl   determinant
    866     871    4 Sl   determinant
    866     872    4 Sl   determinant
```

Один PID объединяет четыре разных TID; NLWP=4. Для лимита 4 в трассировке записаны три успешных clone: дополнительные потоки создаются с флагами CLONE_VM, CLONE_FILES и CLONE_THREAD. Основной поток уже существует до запуска worker.

В этой среде первая попытка clone3 вернула ENOSYS, после чего библиотека успешно использовала clone. Это совместимый запасной путь, а не ошибка вычисления. Вызов futex(...FUTEX_WAIT_BITSET...) наблюдается при ожидании завершения работника. Мьютекс без конкуренции обычно работает в пользовательском пространстве, поэтому каждому lock/unlock не обязан соответствовать системный вызов futex.

В strace видны системные вызовы, а не названия библиотечных pthread_create/pthread_join. Параметр -f включает наблюдение за созданными потоками. exit завершает отдельный поток, exit_group - весь процесс. Файлы фактической трассировки сохранены в report/evidence/.

7. Вывод

Реализована программа на C с POSIX Threads для вычисления определителя рекурсивным разложением. Число потоков ограничивается ключом запуска и включает основной поток. Общая выдача задач защищена мьютексом, результаты работников разделены. Корректность подтверждена независимыми точными проверками; число потоков показано средствами Linux и трассировкой системных вызовов. Эксперимент подтвердил пользу многопоточности на достаточно крупных входах и накладные расходы на малых.

8. Структура файлов

Файл	Назначение
src/determinant.c	Программа, пул, рекурсия, разбор входа.
src/CMakeLists.txt	Сборка и подключение тестов.
src/tests/	Базовые проверки и точный независимый эталон.
src/tools/capture-linux.sh	Демонстрация strace, ps и /proc.
src/tools/benchmark.py	Воспроизводимые замеры и проверка результатов.
src/tools/Dockerfile	Рецепт среды Linux; для текущей проверки использован существующий образ.
examples/	Примеры и входы исследования.
report/	PDF, исходные измерения и свидетельства демонстрации.
DEFENSE.md	Памятка по функциям кода и вопросам на защите.

9. Источники

Задание «Лабораторная работа 3», вариант 9, предоставленное для выполнения работы.

POSIX Threads: https://man7.org/linux/man-pages/man7/pthreads.7.html

clone(2): https://man7.org/linux/man-pages/man2/clone.2.html

futex(2): https://man7.org/linux/man-pages/man2/futex.2.html

strace(1): https://man7.org/linux/man-pages/man1/strace.1.html